

PROYECTO DOPO

MoralesS-RojasH

RETROSPECTIVA CICLO 3

1. ¿Cuáles fueron los mini-ciclos definidos? Justifícalos.

Se definieron 8 mini-ciclos, organizados según las dependencias reales entre la corrección de defectos heredados, la extensión requerida por el problema de la maratón, y las entregas exigidas.

- **Mini-ciclo 1: Refactorización del Ciclo 2.** Se corrigieron dos defectos reales detectados al preparar el algoritmo de la maratón: `spin(wheel, steps)` no soportaba pasos negativos (el bucle interno nunca se ejecutaba con `steps < 0`), y `Wheel.rotate(...)` podía producir un índice negativo por el comportamiento del operador `%` en Java con dividendos negativos. Corregir esto era prerequisite indispensable, ya que el algoritmo de solución necesita rotar en ambas direcciones.
- **Mini-ciclo 2: Constructor `SlotMachine(int n)` y generación aleatoria de símbolos.** Se agregó el constructor que crea una máquina con exactamente `n` ruedas y `n` símbolos (colores tomados al azar de la lista disponible), inicializada aleatoriamente y oculta por defecto, tal como exige el requisito 13.
- **Mini-ciclo 3: Refactor de `distinctSymbols()`.** Se cambió su significado de "símbolos registrados" a "símbolos distintos actualmente visibles", porque es la única fuente de información permitida por el enunciado para que el algoritmo de la maratón sepa qué tan cerca está de ganar, dado que `configuration()` está explícitamente excluida de los métodos permitidos como *testing tool*.
- **Mini-ciclo 4: Mecanismo de cierre y celebración automática.** Se introdujeron `cerrada/isCerrada()/exit()`, el atributo `ganada`, y los métodos `hayJackpot()`, `verificarJackpot()` y `pintarCelebracion()`, separando la consulta pura (`isJackpot()`) de la reacción con efectos secundarios (mensaje y colores), que ahora solo ocurre en la transición de estado, no en cada consulta.
- **Mini-ciclo 5: Algoritmo de solución de la maratón (`SlotMachineContest.resolver`).** Se implementó el algoritmo de 3 fases (distinguir símbolos, descubrir orden relativo, alinear todas las ruedas), usando únicamente `spin(wheel, steps)` y `distinctSymbols()` como fuente de información, según el análisis oficial del problema *Slot Machine* del ICPC World Finals 2025.
- **Mini-ciclo 6: Simulación visual (`simulate`).** Se extendió el algoritmo para que, sobre una máquina visible, pueda observarse el proceso completo hasta el jackpot.

- **Mini-ciclo 7: Pruebas de unidad y pruebas compartidas.** Se construyeron SlotMachineContestTest y las pruebas compartidas correspondientes, cubriendo tanto el camino exitoso como los casos límite ($n < 2$, límite de acciones).
- **Mini-ciclo 8: Pruebas de aceptación y validación final.** Se prepararon las 2 pruebas de aceptación, revisando que reflejaran fielmente el comportamiento real del código, y se documentó este ciclo y los anteriores.

Justificación general: igual que en el Ciclo 2, el orden respeta las dependencias reales de la extensión funcional propiamente dicha (mini-ciclos 5-6), tal como exige el título del ciclo.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

De los 8 ciclos, 8 están completos, implementados y funcionando así como sus respectivas pruebas y las pruebas de aceptación creadas.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Cesar Morales hizo 15 horas y Juan Rojas 15 horas, para un total de 30 horas.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

El mayor logro fue lograr que SlotMachineContest resolviera correctamente el problema real de la maratón (*Slot Machine*, ICPC World Finals 2025) sin depender del estado interno de SlotMachine, usando exclusivamente `spin(wheel,steps)` y `distinctSymbols()`, exactamente como lo exige el problema original interactivo (donde el resolutor nunca ve los símbolos, solo cuenta cuántos son distintos). Esto fue posible porque primero se resolvió el mecanismo de retroalimentación correcto (el refactor de `distinctSymbols()`), lo que permitió construir el algoritmo de 3 fases sobre una base confiable.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue detectar la victoria sin poder consultar el estado interno de la máquina, respetando la restricción del enunciado de no usar `configuration()` dentro del algoritmo de solución. La primera aproximación (detectar el jackpot leyendo `lastMessage()`) resultó frágil, porque una vez alcanzado el jackpot la máquina podía cerrarse sola y sobrescribir ese mensaje con un error en la siguiente llamada. Para resolverlo, se separó claramente la consulta pura del estado (`isJackpot()` -> `hayJackpot()`, sin efectos secundarios) de la reacción a los cambios de estado (`verificarJackpot()`, que solo actúa en la transición, controlada por el nuevo atributo `ganada`), y finalmente `yaGano()` se simplificó a usar directamente `distinctSymbols() == 1`, que es la señal más simple y confiable disponible.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Como equipo, se investigó a fondo el problema real detrás del enunciado (el problema oficial del ICPC World Finals 2025) antes de programar la solución, en vez de improvisar un algoritmo propio.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿Por qué?

La práctica más útil este ciclo fue el refactoring continuo antes de extender: cada vez que se detectaba una limitación real (rotaciones sin soporte para pasos negativos, isJackpot() con efectos secundarios que ensuciaban la detección de victoria), se corregía esa base antes de construir la siguiente capa de funcionalidad sobre ella, en vez de parchear el algoritmo de la maratón alrededor de esas limitaciones.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

- International Collegiate Programming Contest. (2025). *Problem I: Slot Machine*. ICPC World Finals 2025. <https://open.kattis.com/problems/slotmachine>
- International Collegiate Programming Contest. (2025). *2025 ICPC World Finals Solution Sketches*. <https://icpc.global/worldfinals/problems/2025-ICPC-World-Finals/2025-ICPC-Solutions.pdf>
- Oracle. (s.f.). *Class HashSet*. Java Platform SE 17 Documentation. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashSet.html>
- OpenAI. (2023). ChatGPT <https://chat.openai.com/cha>
- Anthropic. (2026). *Claude*. <https://claude.ai>